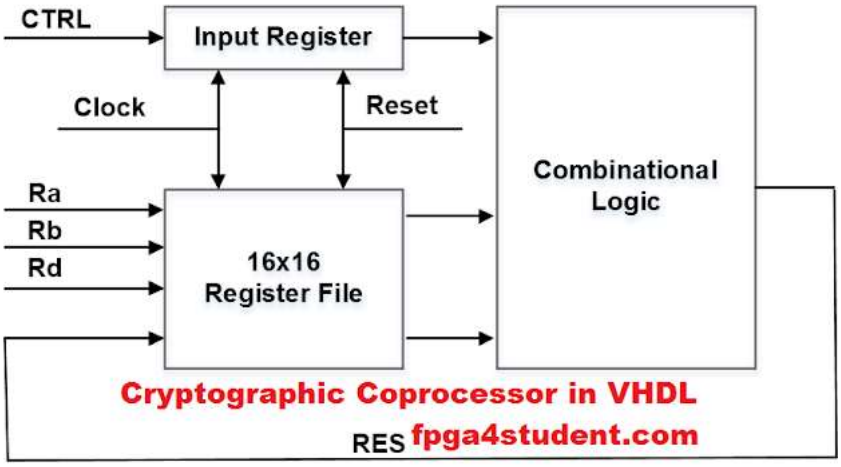
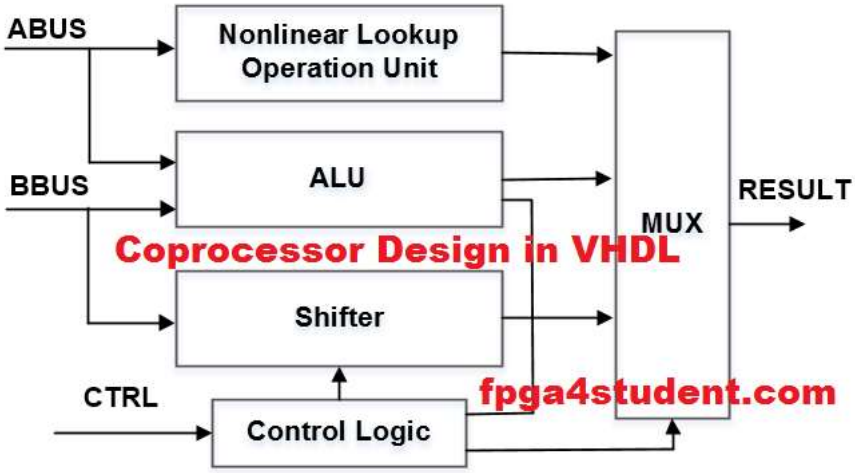
Cryptographic Coprocessor Design in VHDL

In this VHDL project, the complete coprocessor for cryptographic applications is designed and implemented in VHDL.
As mentioned in the previous Verilog/ VHDL projects, the coprocessor provideds standard instructions and dedicated function units specific for security. The co-processor is designed and implemented in VHDL, but the N-bit Adder in ALU unit is implemented in Verilog.
The block diagram of the coprocessor is as follows:



First of all, let's implement the combinational logic unit of the coprocessor. Following is the block diagram of the combinational logic unit:



The major modules of the combinational logic unit such as ALU, Shifting Unit, and Nonlinear Lookup Operation were posted in previous posts:
ALU Design in VHDL
Shifter Design in VHDL
Nonlinear Lookup Implementation in VHDL
The instruction set architecture of the coprocessor is as follows:
0. ADD: ABUS + BBUS -> RESULT
1. SUB: ABUS - BBUS -> RESULT
2. AND: ABUS & BBUS -> RESULT
3. OR: ABUS | BBUS -> RESULT
4. XOR: ABUS ^ BBUS -> RESULT
5. NOT: ~ABUS -> RESULT
6. MOV: ABUS -> RESULT
7. NOP: No Operation
8. ROR8: RESULT<= SHIFTINPUT(7 downto 0)&SHIFTINPUT(15 downto 8) ;

9. ROR4: RESULT<= SHIFTINPUT(3 downto 0)&SHIFTINPUT(15 downto 4) ;
10. SLL8: RESULT<= SHIFTINPUT(7 downto 0) & "00000000";
11. LUT: RESULT <= OUTPUT of LOOKUP TABLE Implementation
VHDL code for combinational logic unit of the coprocessor:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
-- fpga4student.com: FPGA projects, Verilog projects, VHDL projects
-- VHDL project: Cryptographic coprocessor Design in VHDL
-- VHDL code for Combinational Logic unit of the coprocessor
entity structural_VHDL is
port ( A_BUS: in std_logic_vector(15 downto 0);
   B_BUS: in std_logic_vector(15 downto 0);
   CTRL: in std_logic_vector(3 downto 0);
   RESULT: out std_logic_vector(15 downto 0)
  );
end structural_VHDL;

architecture Behavioral of structural_VHDL is
component non_linear_lookup is
port (  LUTIN: in std_logic_vector(7 downto 0);
   LUTOUT: out std_logic_vector(7 downto 0)
 );
end component non_linear_lookup;
component shifter is
  generic ( N: integer:=16
  );
    Port ( SHIFTINPUT : in  STD_LOGIC_VECTOR(N-1 downto 0);
   SHIFT_Ctrl : in  STD_LOGIC_VECTOR(3 downto 0);
   SHIFTOUT: out  STD_LOGIC_VECTOR(N-1 downto 0)
  );
end component shifter;
component ALU is
 port (
   ABUS: in std_logic_vector(15 downto 0);
   BBUS: in std_logic_vector(15 downto 0);
   ALUctrl: in std_logic_vector(3 downto 0);
   ALUOUT: out std_logic_vector(15 downto 0)
   );
end component ALU;
signal tmp_out1,tmp_out2,tmp_out3: std_logic_vector(15 downto 0);
signal lut_out: std_logic_vector(7 downto 0);
begin
-------------
-- ALU Unit--
-------------
ALU_unit: ALU port map( ABUS => A_BUS, BBUS => B_BUS,ALUctrl => CTRL,ALUOUT => tmp
-----------------
-- Shifter Unit--
-----------------
shifter_unit: shifter generic map ( N => 16) -- shifter
 port map( SHIFTINPUT => B_BUS, SHIFT_Ctrl => CTRL,SHIFTOUT => tmp_out2 );
-------------------------------------
-- Non-linear Lookup Operation Unit--
-------------------------------------
non_linear_lookup_unit: non_linear_lookup
 port map( LUTIN => A_BUS(7 downto 0), LUTOUT => lut_out);
 tmp_out3 <= A_BUS(15 downto 8) & lut_out;

------------------------
-- Control Logic Unit--
------------------------
control_logic: process(CTRL,tmp_out1,tmp_out3,tmp_out2) begin
  case(CTRL(3 downto 3)) is
```

```
  when "0" =>
    RESULT <= tmp_out1;
  when others =>
    case(CTRL(1 downto 0)) is
    when "11" =>
     RESULT <= tmp_out3;
    when others =>
     RESULT <= tmp_out2;
    end case;
  end case;
 end process control_logic;

end Behavioral;
```

VHDL code for Register File:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.numeric_std.all;
-- fpga4student.com: FPGA projects, Verilog projects, VHDL projects
-- VHDL project: Cryptographic coprocessor Design in VHDL
-- VHDL code for 16x16 bit Register File( Read/Write Synchronous)
entity register_file is
port ( clock: in std_logic; -- clock
   reset: in std_logic; -- reset
   RdWEn: in std_logic; -- write enable signal
   RES : in std_logic_vector(15 downto 0); -- write value
   Ra,Rb,Rd: in std_logic_vector(3 downto 0); -- selected value of Registers Ra, R
   SRCa,SRCb: out std_logic_vector(15 downto 0) -- read value
  );
end register_file;

architecture Behavioral of register_file is

type mem_type is array(0 to 15) of std_logic_vector(15 downto 0);
signal REG_FILE: mem_type :=( -- memory initialization
  0 => x"0001",
  1 => x"c505",
  2 => x"3c07",
  3 => x"4d05",
  4 => x"1186",
  5 => x"f407",
  6 => x"1086",
  7 => x"4706",
  8 => x"6808",
  9 => x"baa0",
  10 => x"c902",
  11 => x"100b",
  12 => x"c000",
  13 => x"c902",
  14 => x"100b",
  15 => x"B000",
  others => (others => '0')
  );
begin
---------------------------------
-- Write Operation (Synchronous)--
---------------------------------
write_operation: process(clock)
begin
if(rising_edge(clock)) then
 if(RdWEn='1') then -- Write when RdWEn = '1'
  REG_FILE(to_integer(unsigned(Rd))) <= RES;
 end if;
```

```
end if;
end process;
----------------------------------
-- Read Operation (Synchronous)--
----------------------------------
read_operation: process(clock)
begin
if(rising_edge(clock)) then
 if(reset='1') then
  SRCa <= x"0000";
  SRCb <= x"0000";
 else
  SRCa <= REG_FILE(to_integer(unsigned(Ra)));
  SRCb <= REG_FILE(to_integer(unsigned(Rb)));
 end if;
end if;
end process;
end Behavioral;
```

VHDL code for the complete coprocessor:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
-- fpga4student.com: FPGA projects, Verilog projects, VHDL projects
-- VHDL project: Cryptographic coprocessor Design in VHDL
-- Complete Processor in VHDL
entity Co_Processor is
port (
   clock, reset: in std_logic; -- clock and reset
   CTRL: in std_logic_vector(3 downto 0); -- Control Opcode
   Ra, Rb, Rd: in std_logic_vector(3 downto 0) -- Ra, Rb: Source Registers, Rd: De

  );
end Co_Processor;

architecture Behavioral of Co_Processor is
-- Register file in VHDL
component register_file is
port ( clock: in std_logic;
   reset: in std_logic;
   RdWEn: in std_logic;
   RES : in std_logic_vector(15 downto 0); -- write value
   Ra,Rb,Rd: in std_logic_vector(3 downto 0); -- selected value of Registers Ra, R
   SRCa,SRCb: out std_logic_vector(15 downto 0) -- read value
  );
end component register_file;
-- Combinational logic in VHDL
component structural_VHDL is
port ( A_BUS: in std_logic_vector(15 downto 0);
   B_BUS: in std_logic_vector(15 downto 0);
   CTRL: in std_logic_vector(3 downto 0);
   RESULT: out std_logic_vector(15 downto 0)
  );
end component structural_VHDL;
signal Write_Enable: std_logic;
signal read_data1,read_data2,write_data:std_logic_vector(15 downto 0);
signal ctrl_tmp: std_logic_vector(3 downto 0);
signal Rd_tmp: std_logic_vector(3 downto 0);
begin
-- 16x 16-bit Register file in VHDL
Register_file_16x16: register_file port map( clock=> clock, reset => reset  ,
            RdWEn => Write_Enable, RES => write_data, Ra => Ra, Rb => Rb, Rd => Rd
            SRCa => read_data1, SRCb => read_data2);
-- Combinational logic in VHDL
```

```
Combinational_logic: structural_VHDL port map( A_BUS=>read_data1, B_BUS => read_da
             RESULT => write_data);
--- input register VHDL
process(clock,reset) begin
if(rising_edge(clock)) then
 if(reset = '1') then
  ctrl_tmp <= x"0";
  Rd_tmp <= x"0";
 else
  Rd_tmp <= Rd;
  ctrl_tmp <= CTRL;
  if(ctrl_tmp = "0111") then -- NOP Instruction -- DO not allow to write to the Re
   Write_Enable <= '0';
  else
   Write_Enable <= '1';
  end if;
 end if;
end if;

end process;

end Behavioral;
```

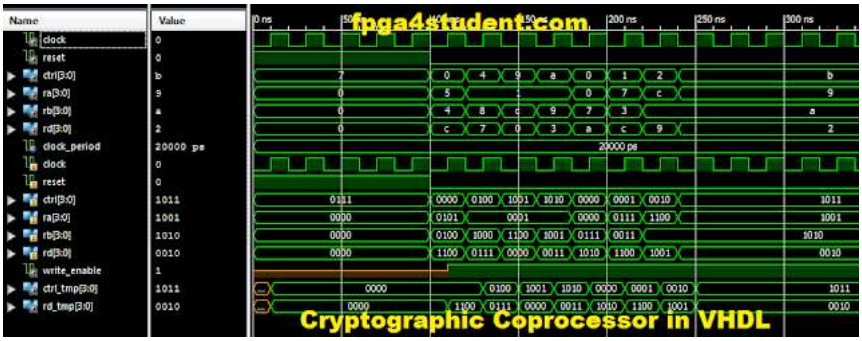
VHDL Testbench code for the cryptographic coprocessor:

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-- fpga4student.com: FPGA projects, Verilog projects, VHDL projects
-- VHDL project: Cryptographic coprocessor Design in VHDL
-- VHDL Testbench Code to verify the coprocessor

ENTITY test_program IS
END test_program;

ARCHITECTURE behavior OF test_program IS
    -- Component Declaration for the coprocessor
    COMPONENT Co_Processor
    PORT(
         clock : IN  std_logic;
         reset : IN  std_logic;
         CTRL : IN  std_logic_vector(3 downto 0);
         Ra : IN  std_logic_vector(3 downto 0);
         Rb : IN  std_logic_vector(3 downto 0);
         Rd : IN  std_logic_vector(3 downto 0)
        );
    END COMPONENT;
   --Inputs
   signal clock : std_logic := '0';
   signal reset : std_logic := '0';
   signal CTRL : std_logic_vector(3 downto 0) := (others => '0');
   signal Ra : std_logic_vector(3 downto 0) := (others => '0');
   signal Rb : std_logic_vector(3 downto 0) := (others => '0');
   signal Rd : std_logic_vector(3 downto 0) := (others => '0');
   -- Clock period definitions
   constant clock_period : time := 20 ns;
BEGIN
 -- fpga4student.com: FPGA projects, Verilog projects, VHDL projects
 -- VHDL project: Cryptographic coprocessor Design in VHDL
 -- Instantiate the coprocessor
   uut: Co_Processor PORT MAP (
          clock => clock,
          reset => reset,
          CTRL => CTRL,
```

```
          Ra => Ra,
          Rb => Rb,
          Rd => Rd
        );
  -- Clock process definitions
  clock_process :process
  begin
 clock <= '0';
 wait for clock_period/2;
 clock <= '1';
 wait for clock_period/2;
  end process;
  stim_proc: process
  begin
     -- hold reset state for 100 ns.
 reset <= '1';
 Ra <= "0000";
 Rb <= "0000";
 Rd <= "0000";
 CTRL <= "0111";
     wait for 100 ns;
 reset <= '0';--- ADD R5,R4, R12
 Ra <= "0101";
 Rb <= "0100";
 Rd <= "1100";
 CTRL <= "0000";
     wait for clock_period;
 Ra <= "0001"; --- XOR R1,R8,R7
 Rb <= "1000";
 Rd <= "0111";
 CTRL <= "0100";
     wait for clock_period;
 Ra <= "0001"; --- ROR4 R12,R0
 Rb <= "1100";
 Rd <= "0000";
 CTRL <= "1001";
     wait for clock_period;
 Ra <= "0001"; --- SLL8 R9,R3
 Rb <= "1001";
 Rd <= "0011";
 CTRL <= "1010";
     wait for clock_period;
 Ra <= "0000"; --- ADD R0,R7,R10
 Rb <= "0111";
 Rd <= "1010";
 CTRL <= "0000";
     wait for clock_period;
 Ra <= "0111"; --- SUB R7,R3,R12
 Rb <= "0011";
 Rd <= "1100";
 CTRL <= "0001";
 --- NOP
-- CTRL <= "0111";
     wait for clock_period;
 Ra <= "1100"; --- AND R12,R10,R9
 Rb <= "1010";
 Rd <= "1001";
 CTRL <= "0010";
 --- NOP
-- CTRL <= "0111";
     wait for clock_period;
 Ra <= "1001"; --- LUT R9,R2
 Rb <= "1010";
```

```
  Rd <= "0010";
  CTRL <= "1011";
      wait;
   end process;

END;
```

Simulation Waveform for the cryptographic coprocessor:



The cryptographic coprocessor in VHDL is completely verified using the VHDL testbench. You may try many different test programs to see how the coprocessor works. Besides observing the simulation waveform, take a look at the memory content of the register file to see how instructions affect the register file.

You may like this:

How to load a text file into FPGA
Verilog code for a Microcontroller
A complete 8-bit Microcontroller in VHDL
Single-cycle MIPS Processor in Verilog
Verilog Code for 16-bit RISC Processor
32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-1)
32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-2)
32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-3)
What is FPGA Programming? FPGA vs Software programming
Recommended and affordable Xilinx FPGA boards for students
Recommended and affordable Altera FPGA boards for students

Recommended VHDL projects:

1. What is an FPGA? How VHDL works on FPGA
2. VHDL code for FIFO memory
3. VHDL code for FIR Filter
4. VHDL code for 8-bit Microcontroller
5. VHDL code for Matrix Multiplication
6. VHDL code for Switch Tail Ring Counter
7. VHDL code for digital alarm clock on FPGA
8. VHDL code for 8-bit Comparator
9. How to load a text file into FPGA using VHDL
10. VHDL code for D Flip Flop
11. VHDL code for Full Adder
12. PWM Generator in VHDL with Variable Duty Cycle
13. VHDL code for ALU
14. VHDL code for counters with testbench
15. VHDL code for 16-bit ALU
16. Shifter Design in VHDL
17. Non-linear Lookup Table Implementation in VHDL
18. Cryptographic Coprocessor Design in VHDL
19. Verilog vs VHDL: Explain by Examples
20. VHDL Code for Clock Divider on FPGA
21. How to generate a clock enable signal instead of creating another clock domain
22. VHDL code for debouncing buttons on FPGA
23. VHDL code for Traffic light controller

24. VHDL code for a simple 2-bit comparator
25. VHDL code for a single-port RAM
22. VHDL code for debouncing buttons on FPGA
23. VHDL code for Traffic light controller
24. VHDL code for a simple 2-bit comparator
25. VHDL code for a single-port RAM
26. VHDL code for Car Parking System using FSM
27. VHDL coding vs Software Programming
28. VHDL code for MIPS Processor
29. VHDL code for Moore FSM Sequence Detector
30. VHDL code for Seven-Segment Display on Basys 3 FPGA

DMCA PROTECTED

Privacy Policy | Disclaimer | Sitemap | Contact | Advertise Here
Copyright © 2016-2020 FPGA4student.com All Rights Reserved.